Design Patterns & Architecture MVC

Cours de Développement d'Applications Informatiques (DAI)

FILIÈRE : BTS DAI

Introduction aux Design Patterns

Définition et enjeux du développement logiciel structuré.

1. Qu'est-ce qu'un Design Pattern ?

Un **Design Pattern** (Patron de Conception) est une solution standardisée et éprouvée à un problème récurrent rencontré lors du développement logiciel.

Avantages :

- Améliore la lisibilité et la maintenance.
- Fournit un langage commun aux développeurs.
- Réduit les bugs architecturaux.

> *"C'est un plan d'architecte réutilisable que l'on adapte à notre contexte spécifique."*

2. Classification des Patterns

Création

Gèrent la manière dont les objets sont instanciés (créés).

Ex: Singleton, Factory.

Structure

Gèrent l'organisation et l'assemblage des classes.

Ex: Adapter, Facade.

Comportement

Gèrent la communication entre les objets.

Ex: Observer, Strategy.

Pourquoi utiliser les Patterns ?

Structure

Organise le code de manière logique et prévisible pour tous les développeurs.

Maintenance

Facilite les modifications futures sans casser les fonctionnalités existantes.

Qualité

Réduit les bugs en utilisant des solutions éprouvées à des problèmes récurrents.

La Séparation des Responsabilités

SoC

Separation of Concerns

Le principe fondamental

Chaque partie du programme doit s'occuper d'une seule tâche :

- ✓ Interface Utilisateur (UI)
- ✓ Logique Métier (Algorithmes)
- ✓ Gestion des Données (BDD)

Architecture MVC : Vue d'ensemble

Model (📦)

Représente les données et la logique métier. C'est le cœur de l'application, indépendant de l'affichage.

View (🎨)

Représente l'interface utilisateur. Elle affiche les données fournies par le Modèle via le Contrôleur.

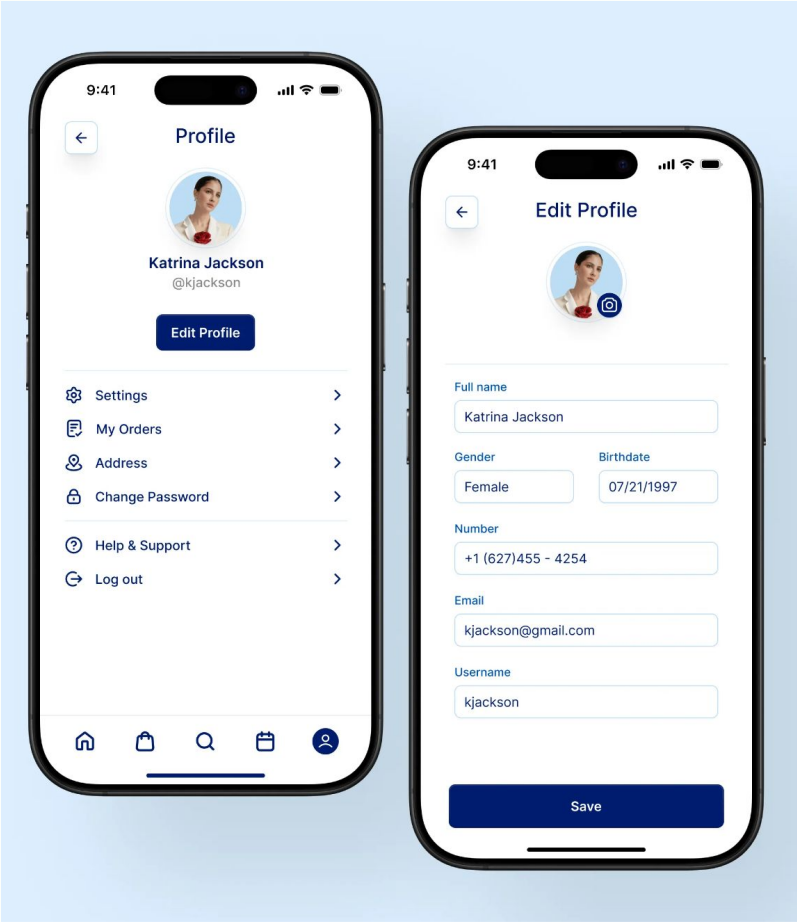
Controller (🎮)

Le médiateur. Il reçoit les actions de l'utilisateur, met à jour le Modèle et rafraîchit la Vue.

Le Modèle

Le Modèle contient les classes de données (POJO). Voici un exemple fonctionnel :

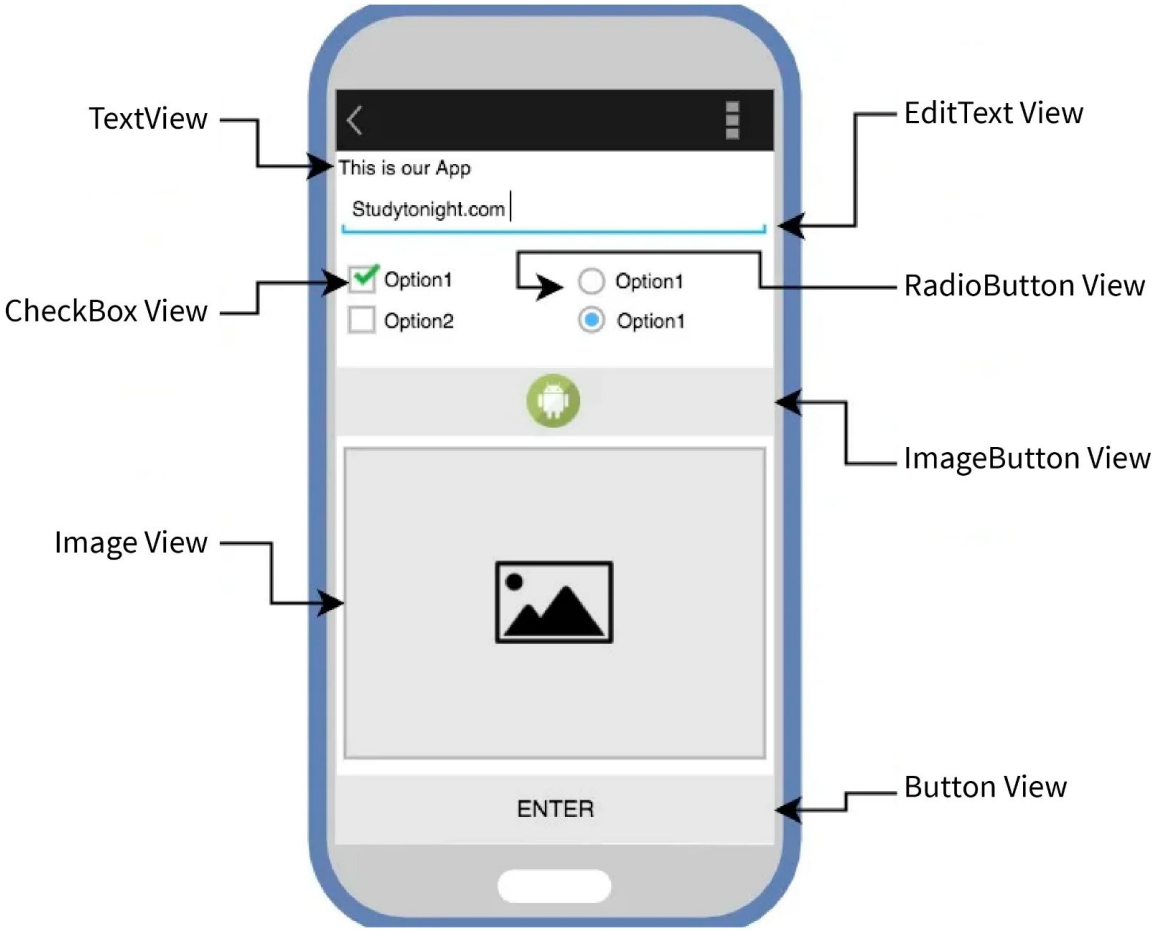
```
public class User {
  private String name;
  private String email;
  public User(String name, String email) {
    this.name = name;
    this.email = email;
  }
  public String getName() {
    return name;
  }
}
```



La Vue

En Android, la Vue est principalement définie en XML.

Elle ne contient aucune logique de décision.



Le Contrôleur

En Android Java, l'**Activity** joue le rôle de Controller.

```
public class MainActivity extends AppCompatActivity {
  private User model;
  private TextView userView;
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    model = new User("Amina", "amina@bts.ma");
    userView = findViewById(R.id.userName);
    userView.setText(model.getName());
  }
}
```

Comparaison des rôles MVC

Composant	Rôle Principal	Exemple Android
Model	Données et Logique	Classes Java (User, Task)
View	Interface Visuelle	layout.xml, Widgets
Controller	Gestionnaire d'événements	Activity, Fragment

Le Flux de données MVC

Action

L'utilisateur clique sur un bouton dans la Vue.

Signal

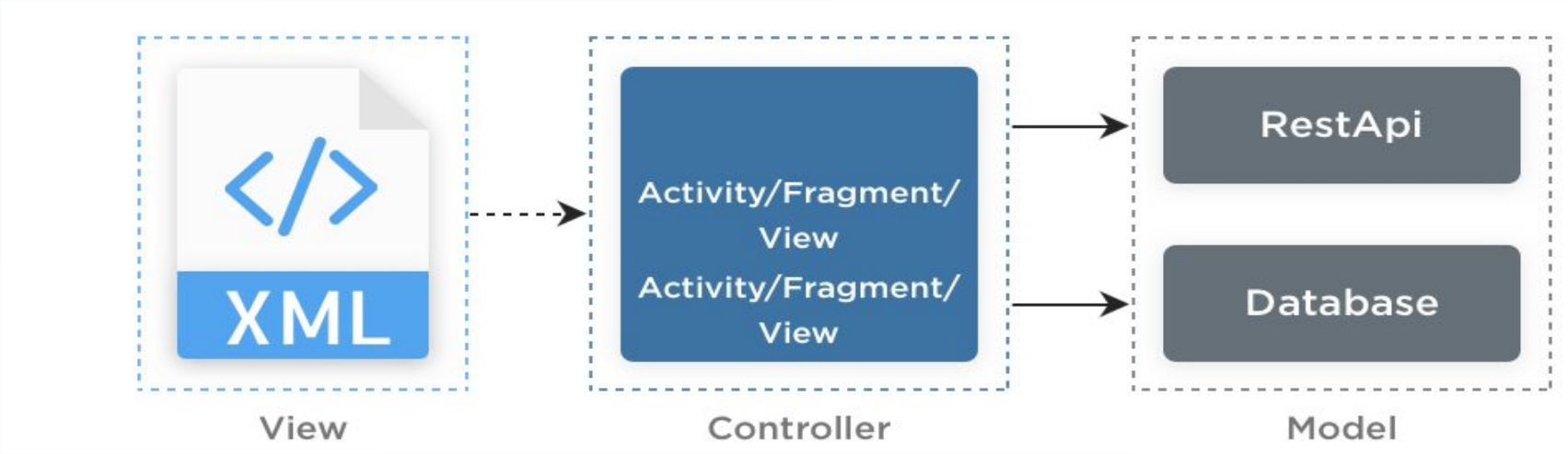
Le Contrôleur intercepte l'événement de clic.

Update

Le Contrôleur modifie les données dans le Modèle.

Refresh

La Vue est mise à jour pour afficher le changement.



Le problème de la "God Activity"

- **Massive Controller :** L'Activity devient trop chargée car elle mélange UI et Logique.
- **Testabilité :** Le MVC Android pur est difficile à tester unitairement car le code est lié à l'OS Android.
- **Evolution :** C'est pour cela qu'on évolue vers le **MVVM** (Model-View-ViewModel) ou la **Clean Architecture**.

Note : Maîtriser le MVC est la première étape indispensable avant d'aborder ces architectures avancées.

L'Architecture MVVM

Model - View - ViewModel

Vers un développement Android moderne et robuste

1. Introduction à MVVM

L'architecture **MVVM** est le standard recommandé par Google. Elle vise à découpler totalement l'interface utilisateur de la logique de présentation.

- 📦 **Model :** La source de vérité (données + API/BDD).
- 🎨 **View :** Activité ou Fragment. N'affiche que ce qu'on lui donne.
- 🧠 **ViewModel :** Le cerveau. Prépare les données pour la View sans la connaître.

Découplage maximum

2. Le rôle clé du ViewModel

Le **ViewModel** est responsable de la conservation et de la préparation des données pour l'interface utilisateur.

Survie

Il survit aux changements de configuration (ex: rotation de l'écran).

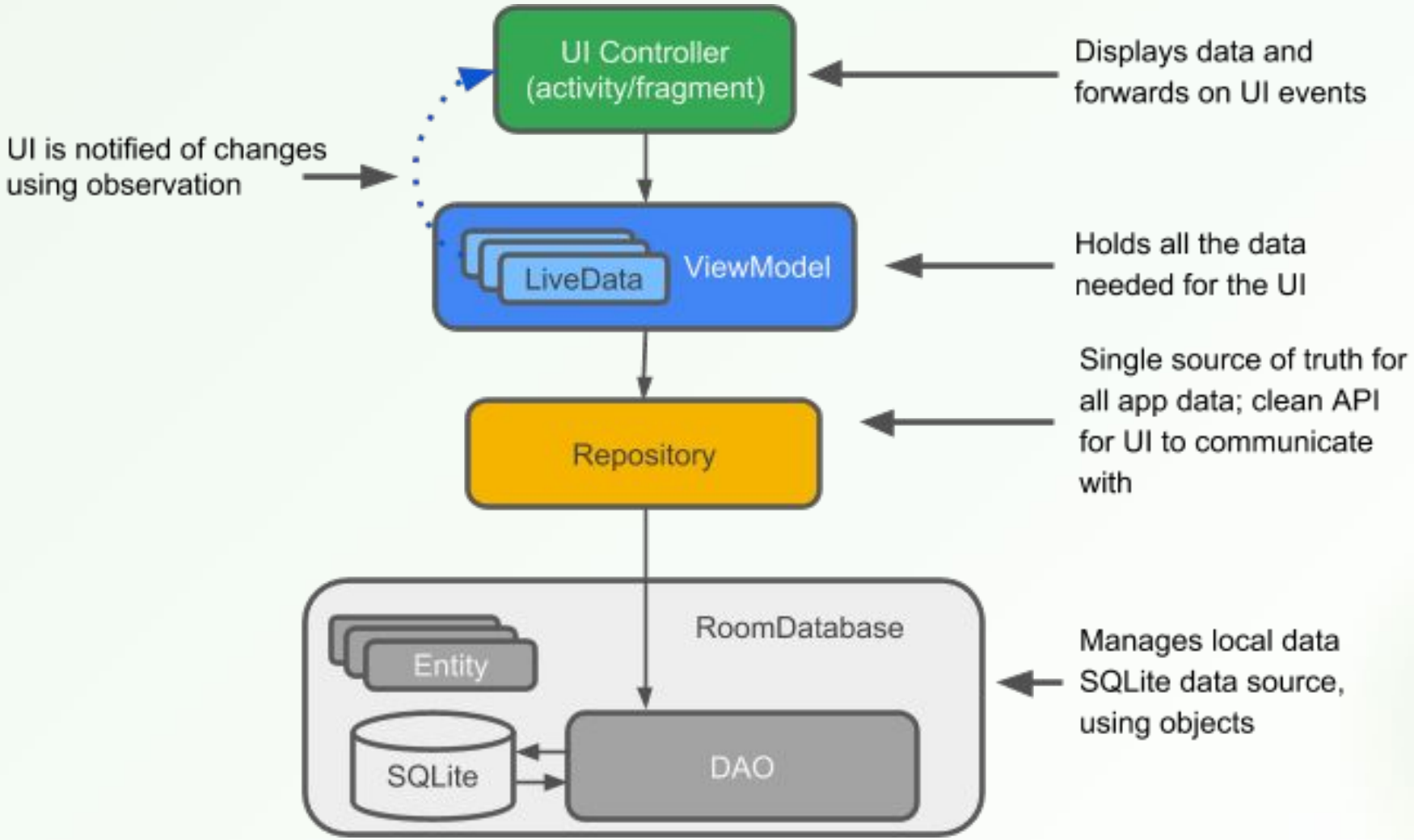
Indépendance

Il ne contient **JAMAIS** de référence à une Activity ou un Fragment.

Passerelle

Il communique avec le Model pour récupérer les données fraîches.

2. Le rôle clé du ViewModel



3. LiveData et Observation

La **LiveData** est une classe de stockage de données "observable" qui respecte le cycle de vie Android.

```
// Dans le ViewModel;
private MutableLiveData name = new MutableLiveData<>
();

public LiveData getName() {
    return name;
}
```

La **View** observe ces données. Dès que la valeur change dans le ViewModel, la View réagit automatiquement.

```
// Dans l'Activity;
viewModel.getName().observe(this, newName -> {
    textView.setText(newName);
});
```

4. Le Flux de données MVVM

1. Action

L'utilisateur clique sur la **View**.

→

2. Traitement

La View prévient le **ViewModel**.

→

3. Données

Le ViewModel demande au **Model**.

→

4. Notification

Le ViewModel met à jour sa **LiveData**.

→

5. Rendu

La **View** (qui observe) se rafraîchit.

5. Comparatif : MVC vs MVVM

Critère	MVC (Ancien)	MVVM (Moderne)
Couplage	Fort (Activity liée au Model)	Faible (ViewModel indépendant)
Testabilité	Moyenne (Besoin de l'OS)	Élevée (Tests unitaires faciles)
Complexité	Simple	Moyenne (LiveData à maîtriser)
Android Recommandé	Non (obsolète)	Oui (Standard Jetpack)

6. Concepts Indispensables

Android Core

- **Activity :** Point d'entrée UI.
- **Fragment :** Portion réutilisable d'UI.
- **Adapter :** Lien données ↔ RecyclerView.
- **Intent :** Communication Inter-Apps.

Architecture Components

- **Lifecycle :** Gestion des états (Start/Stop).
- **Observer :** Pattern de notification.
- **Data Binding :** Liaison XML directe (optionnel).

7. Exemple : Approche MVC

Dans l'approche MVC (Controller surchargé), tout est fait dans l'Activity :

```
// Approche MVC classique;
button.setOnClickListener(v -> {
    String name = editText.getText().toString();
    // Le controller modifie le model;
    user.setName(name);
    // Le controller met à jour la vue manuellement;
    textView.setText(user.getName());
});
```

✕Inconvénient : Si l'écran tourne, les données saisies disparaissent souvent.

8. Exemple : Approche MVVM

Dans l'approche MVVM, la View n'ordonne pas de changement d'affichage, elle **observe** :

```
// Action : La View informe le ViewModel;
button.setOnClickListener(v -> {
    viewModel.setName(editText.getText().toString());
});

// Observation : La View réagit au changement de donnée;
viewModel.getName().observe(this, name -> {
    textView.setText(name);
});
```

✔Avantage : Si l'écran tourne, la LiveData est ré-observée et le texte revient instantanément.

9. Compétences à maîtriser

- Expliquer la différence de couplage entre MVC et MVVM.
- Savoir implémenter une classe ViewModel héritant de Jetpack.
- Utiliser MutableLiveData pour stocker un état.
- Mettre en place un Observer dans une Activity ou un Fragment.
- Éviter de mettre de la logique métier dans les méthodes onCreate().

10. Méthodologie d'apprentissage

Conseils pour réussir ce module :

1. Commencez par faire un projet en **MVC** (le plus intuitif).
2. Provoquez un bug (ex: rotation d'écran) ou surchargez votre Activity.
3. Introduisez alors **MVVM** comme le "remède" pour assainir votre code.

C'est à ce moment précis que la théorie prend tout son sens pratique.

Des questions ?

Préparez-vous pour le TP d'implémentation MVVM avec LiveData.

FILIÈRE : BTS DAI - DÉVELOPPEMENT ANDROID